

TP Test Unitaire et Pointeurs Dynamiques

Ce TP a pour objectif de découvrir les tests unitaires en C/C++ avec Visual Studio et Google Test. Vous apprendrez à écrire des tests, les exécuter, comprendre les erreurs et corriger le code. Une partie explicative est incluse pour comprendre comment fonctionne Google Test et comment écrire vos tests.

Créer un dossier TP4_TU dans votre dossier Git.

Copié-collé le fichier .gitignore fourni dans ce dossier.

Dans un terminal taper (ouvert dans le dossier TP4_TU) :

```
git add .gitignore
git commit -m « ajout gitignore »
git push
```

Vérifier dans Github que votre commit a bien été effectué.

Vous êtes prêt à attaquer le TP.

Commitez après l'écriture de chaque test et après chaque correction de bug !

Théorie : comprendre les tests unitaires

Les tests unitaires permettent de tester chaque fonction d'un programme unitairement. Cela permet de vérifier que notre application fonctionne correctement à tout moment du développement. Si une régression (problème qui n'existait pas auparavant) apparaît, on est directement informé.

Un test unitaire vérifie qu'une fonction produit le bon résultat pour un ensemble d'entrées.

Propriétés importantes :

- Automatique
- Reproductible
- Rapide
- Indépendant

Il existe plusieurs framework de tests en C. Nous allons utiliser CppUnit.

Avant de voir cela, quelques notions.

1. Macros

Un **macro** est un bout de code que le préprocesseur remplace par un autre fragment de code avant que le compilateur ne commence à compiler.

Les macros sont principalement définies à l'aide de la directive du préprocesseur **#define**.

Exemples :

```
#define PI 3.14159
```

Le préprocesseur remplacera PI, partout dans le code où il le trouvera, par 3.14159, avant que le code soit compilé.

```
#define MAX(a,b) ( (a)> (b) ? (a) : (b) )
```

Cette macro renvoie le maximum entre les deux arguments passés en paramètre.

Si dans votre code vous utilisez MAX(x,y) le préprocesseur remplacera par ((x)>(y) ? (x) : (y))

2. Assertion

Une assertion est une instruction utilisée pour vérifier que des conditions spécifiques sont respectées à un moment donné dans le code.

Si ce que l'assertion évalue renvoie « faux » alors le programme signale une erreur ou peut se terminer.

Afin d'effectuer nos tests unitaires, nous allons créer une macro pour faire les assertions :

```
1. #define mu_assert(message, test) do { if (!(test)) return message; } while (0)
```

Que fait cette macro à votre avis ?

Tests dans Visual Studio

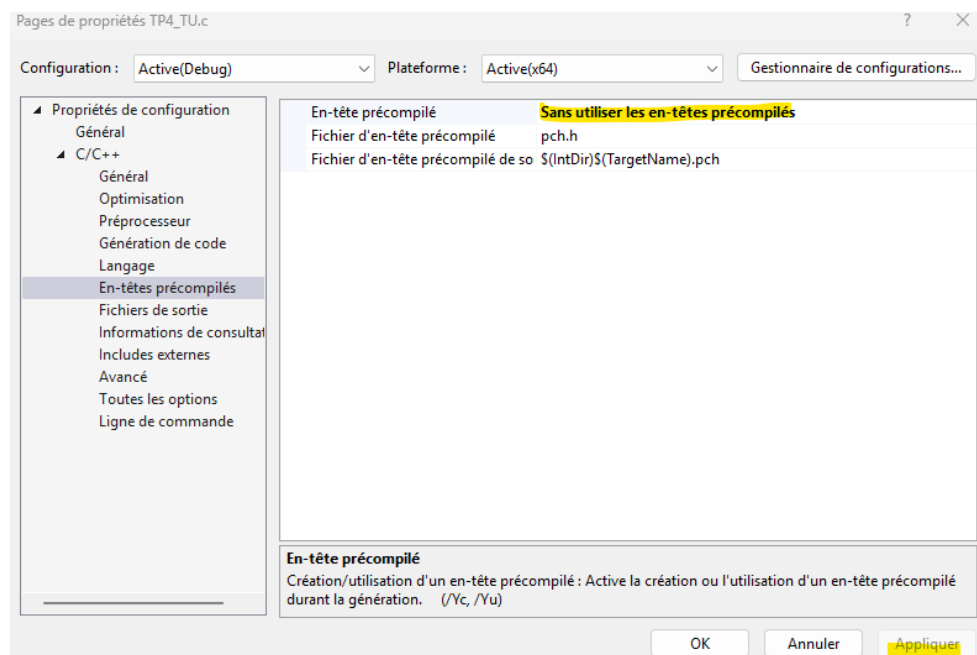
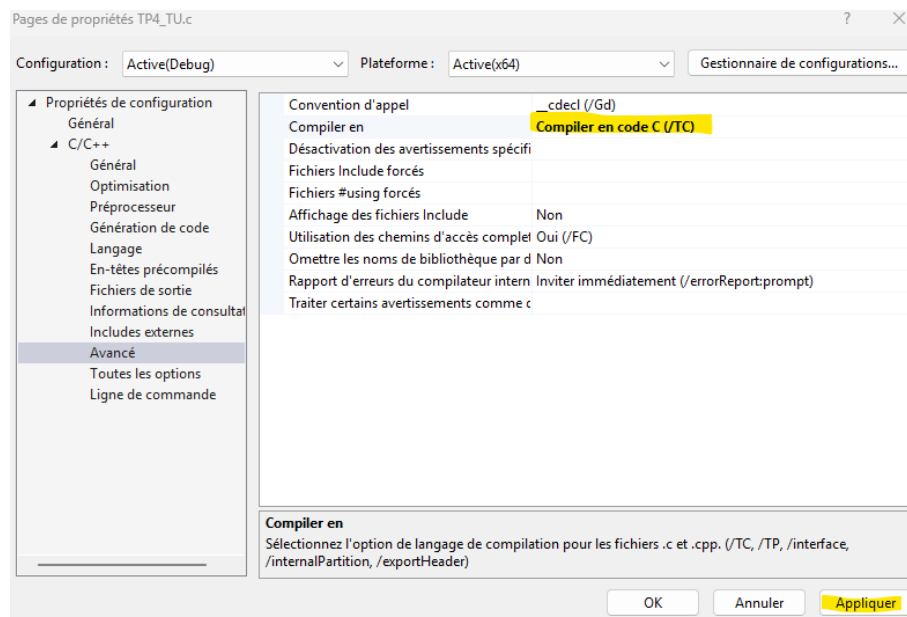
Étapes :

1. Créer projet C++
2. Ajouter projet de test unitaire



Appeler le projet TP4_TU_test

3. Ajouter votre fichier .c au projet de test et vérifier ses propriétés :



4. Générer la solution
5. Ouvrir Test Explorer

1. Créer un premier fichier de test

Dans le projet TP4_TU_test, nous allons compléter le fichier nommé test_TP4_Tu_test.cpp

```
1. #include "pch.h"
2. #include "CppUnitTest.h"
3.
4. using namespace Microsoft::VisualStudio::CppUnitTestFramework;
5.
6. namespace TP4TUTests
7. {
8.     TEST_CLASS(TP4TUTests)
9.     {
10.    public:
11.
12.        TEST_METHOD(TestMethod1)
13.        {
14.        }
15.    };
16. }
```

Comme nous voulons tester du code C, il va falloir rajouter, juste après les include :

```
extern "C" {
#include "../TP4_TU/TP4_TU.h"
}
```

2. Premier test simple

Écrire un premier test qui vérifie que estPair(4) vaut 1.

Exemple de structure attendue :

```
TEST_METHOD(TestEstPair)
{
    Assert::AreEqual(1, estPair(6));
}
```

Exécuter le test dans **Test Explorer**.

Observer s'il passe ou échoue.

3. Tester estPair

Écrire les tests suivants :

1. estPair(4) doit être vrai (1)
2. estPair(7) doit être faux (0)
3. estPair(0) doit être vrai
4. estPair(-2) doit être vrai
5. estPair(-3) doit être faux

Questions

1. Tous les tests passent-ils ?
2. Si non, que peut-on en déduire sur l'implémentation ?
3. Corriger ensuite la fonction.

4. Tester max2

Écrire des tests pour vérifier :

1. max2(5, 3) retourne 5
2. max2(3, 5) retourne 5
3. max2(-1, -8) retourne -1
4. max2(4, 4) retourne 4

Questions

1. Quel test révèle le bug le plus clairement ?
2. Corriger la fonction.

5. Tester factorielle

Écrire des tests pour vérifier :

1. factorielle(0) vaut 1
2. factorielle(1) vaut 1
3. factorielle(4) vaut 24
4. factorielle(5) vaut 120

Puis réfléchir au cas :

5. factorielle(-3)

Questions

1. Le comportement sur les entiers négatifs doit-il être accepté ?
2. Si non, quelle stratégie adopter ?

6. Tester contientMajuscule

Écrire des tests pour vérifier :

1. "bonjour" retourne 0
2. "BonJour" retourne 1
3. "ABC" retourne 1
4. "" retourne 0

Puis réfléchir au cas :

5. pointeur nul : NULL

Travail demandé

Modifier la fonction pour que :

- si la chaîne est null, la fonction retourne 0.

Ajouter le test associé.

7. Tester `divisionExacte`

Écrire des tests pour vérifier :

1. `divisionExacte(10, 2)` retourne 1
2. `divisionExacte(10, 3)` retourne 0
3. `divisionExacte(0, 5)` retourne 1

Puis tester le cas :

4. `divisionExacte(10, 0)`

Question

Pourquoi ce cas est-il dangereux ?

Travail demandé

Modifier la fonction pour que :

- si `b == 0`, la fonction retourne 0.

Ajouter le test associé.

8. Tester `sommeTableau`

Écrire des tests pour vérifier :

1. tableau `{1, 2, 3}` de taille 3 → somme 6
2. tableau `{5}` de taille 1 → somme 5
3. tableau `{-1, 4, -3}` de taille 3 → somme 0
4. tableau vide de taille 0 → somme 0

Puis réfléchir au cas :

5. pointeur nul avec taille 0
6. pointeur nul avec taille strictement positive

Travail demandé

Choisir la convention suivante :

- si `taille <= 0`, retourner 0
- si `tab == null`, retourner 0

Modifier la fonction et ajouter les tests correspondants.

Tests Unitaires pour le BlindTest

Créer un nouveau projet VS dans votre dossier TP

Ajouter les fichiers .c et .h existant au projet

Créer un nouveau projet GoogleTest à votre solution

Faites les tests unitaires du blindtest.

Commitez à toutes les étapes.